Проект atomic: полный отчёт от 6-го семестра до заморозки системы

Шелманова Ева Олеговна, группа Б23-605, НИЯУ МИФИ

Состояние на 25.10.2026 (версия v1.0-freeze)

Этот отчёт рассказывает всю историю проекта подряд: каким был ассистент в 6-м семестре, почему его пришлось развивать, что сделано на каждом этапе, какие проблемы встретились, как их решали и какие получились результаты. Он написан так, чтобы его мог прочитать человек, не знакомый с проектом и не являющийся специалистом по машинному обучению. Специальные слова объясняются при первом появлении. Короткий словарь — в конце, в приложении А.

Для точных технических подробностей есть отдельные документы в папке `docs/`: журнал работ (`journal.md`), реестр решений с номерами D-001… D-044 (`decisions.md`), документы этапов (`stage-A-…`, `stage-B-…`, `stage-C-…`, `stage-E-…`). В тексте ниже номера решений указаны в скобках, чтобы можно было быстро найти подробности.

Состояние на 25.10.2026: система заморожена (тег `v1.0-freeze`), финальные числа получены и воспроизведены. Осталась только письменная часть: отчёт по практике и текст ВКР.

Содержание

1) Коротко о главном

2) С чего всё началось: ассистент 6-го семестра

3) Почему его нужно было развивать: замечания и новые требования

4) План работ: дорожная карта

5) Подготовка: разбор репозитория

6) Данные, с которыми работает система

7) Этап A: модель данных

8) Этап B: методы

9) Этап C: интеграция, эксперименты и исправление слабых мест

10) Этап E: доводка, проверка воспроизводимости и заморозка

11) Итоговые результаты и что они означают

12) Ограничения и честные оговорки

13) Что осталось сделать

14) Приложения: словарь, хронология, где что лежит, как всё запустить

1. Коротко о главном

Что было. В 6-м семестре был сделан прототип ассистента для оператора АЭС. Он отвечал на вопросы по базе знаний с помощью языковой модели (это подход RAG: сначала найти подходящие фрагменты документов, потом сформулировать ответ по ним). Ассистент умел маршрутизировать запросы между тремя «агентами»: поиск по базе знаний, выполнение кода, рассуждение. Проверки ответов как отдельной работающей подсистемы в нём не было: она была описана в отчёте, но в коде отсутствовала.

Что стало. Теперь ассистент дополнен диагностическим контуром. Он смотрит на показания приборов станции (временные ряды из открытого набора данных NPPAD) и по ним:

- строит несколько гипотез о том, что происходит на станции, например: «вероятна течь теплоносителя в горячей нитке, уверенность 65 %»; гипотезы строятся всего по шести основным приборам, теми же, что оператор видит в первую очередь;
- независимо проверяет каждую гипотезу по всем 82 доступным приборам и собирает свидетельства «за» и «против» со ссылками на конкретный прибор и момент времени;
- проверяет, похожа ли ситуация вообще на что-то известное, и честно отвечает «незнакомая ситуация», если нет;
- подбирает похожие реальные события из международной базы инцидентов на АЭС (ETHZ) и справку о том, как обычно развивается такой сценарий;
- выпускает ответ оператору только после проверки правилами. Правила, а не языковая модель, решают, можно ли показать ответ как есть, нужна ли пометка «требуется оценка оператора» или ответ нужно заблокировать.

Система ничем не управляет и не советует, что делать. Она только интерпретирует данные; решение всегда за оператором.

Главные результаты на данных, которые система никогда не видела при настройке:

- в 95 % моментов времени система ставит диагноз, и он верен в 99,9 % случаев;
- на семи типах ситуаций, которых не было в обучении, она ни разу не поставила ложный диагноз;
- ни один из 1680 специально испорченных ответов (без ссылок, со ссылкой «в никуда», с указанием к действию) не был выпущен;
- на один момент времени уходит около 26 миллисекунд;
- повторный запуск и сборка всего с нуля на «чистом» компьютере дают в точности те же числа.

2. С чего всё началось: ассистент 6-го семестра

2.1. Задача 6-го семестра

Учебная практика 6-го семестра называлась «Разработка интерактивного ассистента для операторов АЭС». Объект — информационный поток и документация, с которыми работает оператор: процедуры, аварийные инструкции, регламенты, справочные таблицы. Нужно было сделать прототип системы, которая быстро находит нужные фрагменты регламентов, отвечает со ссылками на источники и проверяет ответы по правилам. В требованиях стояли достоверность (ссылки в каждом ответе), трассируемость (журнал всех операций), качество поиска, модульность, безопасность интерфейса и воспроизводимость.

2.2. Что было построено

Прототип получил название **atomic**. Его устройство:

- **Точки входа.** Консольный чат (`main.py`) и веб-интерфейс программирования (API) на FastAPI: адрес `POST /chat` принимает вопрос и возвращает ответ, `GET /health` показывает, что сервис жив.
- **Оркестратор `AtomicWorkflow`.** Принимает запрос, передаёт его по цепочке агентов, сохраняет диалог в памяти.
- **Маршрутизатор `RouterAgent`.** Языковая модель относит вопрос к одному из типов: поиск по базе знаний, выполнение кода, анализ, общий вопрос.
- **Агенты.** `RetrieverAgent` ищет в базе знаний и отвечает по найденному. `CommandAgent` пишет и выполняет код Python в отдельном процессе, например для расчёта. `AnalystAgent` рассуждает и анализирует. Все агенты устроены по общему образцу (`BaseAgent`) и выстроены цепочкой: запрос проходит по ней, пока какой-то агент не возьмётся за него.

– **Память и поиск.** История диалога хранится в `ChatMemory`. База знаний лежит в PostgreSQL с расширением pgvector: каждый фрагмент текста превращается в вектор чисел (эмбеддинг), и поиск идёт по близости векторов. Для работы без базы данных есть упрощённая память прямо в программе.
– **Модели.** Все языковые модели работают локально через Ollama, без отправки данных в интернет. Маршрутизатор и аналитик — qwen2.5-7B, поиск и выполнение кода — deepseek-r1, эмбеддинги — nomic-embed-text.
– **Наблюдаемость.** Langfuse записывает трассы: что спросили, какие модели вызывались, что они ответили.
– **Инфраструктура.** Docker Compose поднимает базу данных и Langfuse. Зависимости ведёт менеджер пакетов uv.
– **Тесты.** pytest в двух режимах: быстрый (детерминированные проверки, около минуты) и полный, где качество ответа оценивает другая языковая модель (подход LLM-as-a-Judge, библиотека deepeval, метрика «релевантность ответа»).

2.3. Чего не хватало

Отчёт 6-го семестра описывал подсистему верификации: проверку источников, чисел, логической согласованности. Он также упоминал запасной поиск BM25 (классический алгоритм поиска по словам). Но когда в 7-м семестре открыли код, выяснилось:

– отдельного модуля проверки в коде нет, проверка была описана словами;
– BM25 в коде тоже нет; упрощённая память искала простым совпадением подстрок;
– агенты не выдают структурированных утверждений со ссылками, только текст;

– данных о состоянии станции (телеметрии) нет вообще: ассистент работал только с документами;
– база знаний крошечная: 4 текстовых файла, всего 2,6 КБ, и все про устройство самой ИТ-системы, а не про АЭС.

Всего в прототипе было около 1700 строк кода на Python.

3. Почему его нужно было развивать: замечания и новые требования

3.1. Замечание о верификаторе

Главное замечание к работе 6-го семестра звучало так: верификатор описан, но не существует как система. В нём нет модели доверия к источникам, нет формальных правил, нет механизма, который принимает решение о выпуске ответа. Если же проверку сделать «ещё одной языковой моделью», это не проверка, а иллюзия второго мнения: у неё тот же класс риска, что и у основной модели, только с другой формулировкой запроса. Для АЭС это неприемлемо.

В материалах 7-го семестра был предложен минимальный каркас настоящей проверки:

- **утверждение (Claim)** — одна проверяемая фраза ответа;
- **ссылка на свидетельство (EvidenceRef)** — откуда это взято: документ, фрагмент базы знаний, результат расчёта, показание прибора;
- **набор правил (RulePack)** — простые автоматические проверки вроде «у каждого утверждения есть ссылка» или «ссылка указывает на существующий документ»;
- **решение политики (PolicyDecision)** — итог: выпустить, заблокировать или передать человеку, плюс запись, почему.

Ключевая мысль: окончательное «да» или «нет» говорит не языковая модель, а детерминированный механизм правил. Модель в лучшем случае помогает разбить ответ на утверждения.

3.2. Документ «Upgrade to Guided Operations»

Второй опорный документ описывает, как такая система может развиваться: пять этапов от «информационного слоя» (ответы по документам)

до «управляемых операций». Он же задаёт жёсткие инварианты, то есть правила, которые нельзя нарушать никогда:

– система не управляет оборудованием;
– человек всегда в контуре принятия решений;
– ничего не исполняется автоматически;
– доступ к данным станции только на чтение;
– объяснимость обязательна.

В анализе опасностей документа есть две, которые прямо касаются диагностики: **H3** — система неверно классифицирует ситуацию, **H4** — система слишком уверена в себе. Защита от них — выдавать несколько гипотез вместо одной и калибровать уверенность, то есть добиваться, чтобы «уверенность 80 %» действительно означала правоту примерно в 80 % случаев.

Диплом покрывает этап 1 (контекстный поиск по документам, уже был), этап 2 («мягкая интерпретация»: гипотезы о ситуации с уверенностью) и пассивную часть этапа 3 (ссылки на похожие исторические события без предписания действий). Остальное — перспектива за пределами диплома.

3.3. Задание на производственную практику

Тема практики 7-го семестра: «Интерактивный ассистент для поддержки принятия решений операторами АЭС на основе формирования и верификации гипотез о состоянии системы». Задание содержит 13 технических требований:

№	Требование
1	Сохранить работоспособность базового ассистента
2	Загружать и нормализовать временные ряды NPPAD и структурированные данные
3	Единая семантическая модель диагностической информации
4	Гипотезы по ограниченному набору разрешённых каналов
5	Генератор не видит скрытых данных и истинного ответа
6	Независимая проверка гипотез по полному набору каналов
7	Свидетельства «за» и «против», ссылки на аналогичные события
8	Передача гипотез в конвейер ассистента
9	Детерминированная проверка итогового ответа

№	Требование
10	Трассируемость и аудит
11	Компонентное и сквозное тестирование
12	Оценка качества генерации и проверки (benchmark)
13	Воспроизводимость

4. План работ: дорожная карта

В октябре 2026 года была составлена техническая дорожная карта диплома (`books/Бакалаврский диплом техническая дорожная карта.pdf`). Она делит работу на шесть этапов:

Этап	Содержание	Срок по плану	Фактически
A	Модель данных: загрузка NPPAD, контракты, граф знаний, информационный барьер	31.10.2026	выполнен 24.10.2026
B	Методы: генератор и верификатор гипотез, аналоги, верификатор ответа	декабрь 2026	выполнен 24.10.2026
C	Интеграция с ассистентом, трассировка, сквозные тесты, benchmark	декабрь 2026	выполнен 25.10.2026
D	Отчёт по практике (ГОСТ 7.32) и защита	22–28.12.2026	впереди
E	Доводка, финальный benchmark, заморозка	январь–февраль 2027	выполнен досрочно 25.10.2026
F	Преддипломная практика, текст ВКР, защита	весна 2027	впереди

Этап E выполнен раньше этапа D сознательно: чтобы писать отчёт и диплом на окончательных числах, а не переписывать их потом.

Каждое из 13 требований закреплено за модулем и этапом. Статус по каждому ведётся в `docs/traceability.md`; сейчас все 13 выполнены.

5. Подготовка: разбор репозитория

5.1. «Слоёный» репозиторий

В начале работы папка проекта выглядела запутанно: несколько копий отчётов в разных форматах (.pages, .pdf, .docx) в корне и в подпапках `final_version/` и `books/`, корневые файлы проекта (`main.py`, `pyproject.toml`, `README`) почему-то лежали в папке `unknown/`. Было непонятно, какая версия последняя.

Сделано:

– проверено, что файлы из `unknown/` байт-в-байт совпадают с последним коммитом в git, и они возвращены на место;
– установлено, что версия кода одна: ассистент 6-го семестра (коммит a3f0e16 от 30.03.2026), других копий нет;
– последний отчёт 6-го семестра — `final_version/final_report (2) (2).pdf`;
– найдены расхождения между отчётом и кодом (верификатор и BM25, см. раздел 2.3).

5.2. Порядок в git

– Базовая версия помечена тегом `v0.1-baseline`. Это точка отсчёта для требования 1: с ней всегда можно сравнить, не сломалось ли старое поведение (D-001).
– Вся разработка ведётся в отдельной ветке `diploma`. Каждый законченный шаг — отдельный коммит, каждый этап — свой тег: `v0.2-stage-a`, `v0.3-stage-b`, `v0.4-stage-c`, `v1.0-freeze`.
– Бинарные файлы (отчёты в .pages, .pdf, .docx) и большие данные исключены из git (D-002). Данные восстанавливаются скриптами, а в git лежат только код, конфигурация и тексты.
– Быстрые тесты базовой версии прошли 24.10.2026: это зафиксированная точка «до изменений».

5.3. Удаление вопросов оппонентов из GitHub

В истории репозитория на GitHub оставались файлы с вопросами и ответами оппонентов. Они были удалены через веб-интерфейс, но в истории коммитов сохранились, и в одном из них был пароль локального Langfuse. Что сделано (D-037):

– полная резервная копия репозитория (`~/Documents/atomic-git-backup-2026-10-24.bundle`; в ней эти файлы есть, хранить её нужно только локально);
– история всех веток переписана утилитой `git filter-repo` так, будто этих файлов никогда не было; сами файлы оставлены на диске и добавлены в `.gitignore`;
– хэши коммитов после этого изменились, ссылки в документации обновлены;
– переписанная ветка `main` отправлена на GitHub (это сделали вы сами командой с `--force-with-lease`, потому что такую операцию система прав мне выполнить не дала); проверено, что на GitHub одна ветка, и ни в одном её коммите файлов нет.

Пароль локального Langfuse стоит сменить: он какое-то время был в публичной истории.

6. Данные, с которыми работает система

6.1. NPPAD: показания приборов во время аварий

Что это. NPPAD — открытый набор данных, опубликованный в журнале Scientific Data в 2022 году (Qi B. и соавторы). Он получен на симуляторе PCTRAN — компьютерной модели энергоблока с реактором PWR (водо-водяной реактор под давлением, близкий к ВВЭР). В каждом прогоне симулятор начинает с работы на 100 % мощности, через 0,5 секунды вводит неисправность заданной тяжести и записывает показания 96 «приборов» каждые 10 секунд.

Что	Сколько
Классов ситуаций	18 (17 аварий и нормальная эксплуатация)
Прогонов (отдельных симуляций)	1217
Каналов (приборов)	96
Шаг записи	10 с
Точек времени всего	492 709

Прогон — это одна симуляция: например, «течь теплоносителя в горячей нитке площадью 37 % от 100 см²». У двенадцати классов есть по 99–110 прогонов разной тяжести (от 1 до 100 %), у шести — всего по одному.

Классы ситуаций (коды как в NPPAD):

Код	Что происходит
LOCA	течь теплоносителя первого контура в горячей нитке (трубопровод от реактора к парогенератору)
LOCAC	то же в холодной нитке (трубопровод от парогенератора к реактору)
SGATR, SGBTR	разрыв трубки парогенератора A или B (теплоноситель первого контура попадает во второй)
SLBIC, SLBOC	разрыв паропровода внутри или вне защитной оболочки
FLB	разрыв питательной линии
LLB	разрыв линии продувки во вспомогательном здании
LR	сброс нагрузки (резко падает потребление энергии турбиной)
RW, RI	неконтролируемое извлечение или введение регулирующих стержней
MD	разбавление борной кислоты в теплоносителе (меняет реактивность)
ATWS	переходный процесс без срабатывания аварийной защиты

Код	Что происходит
LACP	потеря электропитания переменного тока
LOF	потеря расхода (заклинивание насоса)
SP	горение водорода
TT	отключение турбины
Normal	нормальная эксплуатация

Откуда взяли. Репозиторий авторов на GitHub (`thu-inet/NuclearPowerPlantAccidentData`, версия от 25.08.2026). Полная копия занимает 16 ГБ, в основном из-за исходных баз симулятора в формате .mdb. Проверили, что такие же данные есть в виде CSV-таблиц для всех прогонов, и удалили .mdb, чтобы освободить место (диск был заполнен на 97 %). Журналы событий симулятора («Transient Report») сохранили: из них берутся тяжесть аварии и последовательность срабатывания систем защиты (D-003).

Что нашли при разборе.

- В 25 файлах одного класса (SLBIC) есть три лишних столбца, которых нет в документации. Их явно отбрасываем, а любой другой неожиданный столбец считается ошибкой (D-005).
- Порядок столбцов в файлах не совпадает с документацией. Загрузчик расставляет их по именам.
- Семь каналов не меняются вообще ни в одном обучающем прогоне (в основном параметры тяжёлой аварии с расплавом).
- 18 прогонов класса RI короче двух минут, из них не получается ни одного «окна» наблюдений.
- У RI тяжесть записана отрицательными числами (−1…−100). Сначала из-за этого неправильно читалась тяжесть; ошибка найдена и исправлена (D-013).

6.2. Аудит данных: «пустые» классы

На этапе C выяснилось, что некоторые классы NPPAD на самом деле почти ничего не содержат. Каждый прогон сравнили с прогоном, где

неисправность введена с долей 0 % (станция просто работает в стационаре). Подробности в `docs/data-audit.md`.

- **MD (100 прогонов)** полностью совпадает со стационаром при любой заявленной тяжести. Концентрация бора во всех прогонах постоянна — 200 ppm. Неисправность в выходных данных симулятора просто не проявилась.
- **SLBOC (100 прогонов)** по теплогидравлике (давления, температуры, расходы) не отличается от стационара. Меняются только мониторы радиации паропровода и конденсатора.
- **ATWS и SP** — неисправность с долей 0 %, станция в стационаре.
- **Normal** — не стационар: в начале прогона закрывается клапан питательной воды.

Что с этим сделали (D-034). MD убрали из классов, на которых система учится: учить систему распознавать «ничего не происходит» как аварию бессмысленно. На этапе B система «распознавала» MD в 78 % случаев, и теперь понятно, что это был артефакт. Прогоны MD, ATWS и SP стали эталоном «события нет»: на них система должна отвечать «отклонений не обнаружено». SLBOC оставили, но его слабость (заметен только по радиации) записали в ограничения.

Итого обучаемых классов — 11: FLB, LLB, LOCA, LOCAC, LR, RI, RW, SGATR, SGBTR, SLBIC, SLBOC.

6.3. «Только один прогон» — что это значит и можно ли улучшить

Шесть классов (ATWS, LACP, LOF, Normal, SP, TT) представлены единственной симуляцией без параметра тяжести. На одном примере нельзя ни научить систему, ни честно её проверить: если поделить один прогон на части, соседние куски попадут и в обучение, и в проверку, и результат будет завышен.

Улучшить ситуацию по-настоящему можно только новыми симуляциями в PCTRAN: на Figshare лежат те же данные, а симулятор у нас

недоступен. Синтетически «размножать» данные (аугментацию) не стали: это означало бы выдавать искусственные примеры за реальные. Вместо этого однопрогонные классы стали полезными **пробами**: на ситуациях, которых система никогда не видела, проверяется, что она не ставит ложный диагноз, а честно говорит «незнакомая ситуация» или «отклонений нет».

6.4. ETHZ: база реальных инцидентов на АЭС

База ETH Zurich Curated Nuclear Events Database (Ayoub A. и соавторы, 2021) содержит описания 1561 реального события на ядерных объектах мира с классификацией по типу исходного события, системе, компоненту и цепочке отказов.

Распространяется она как книга Excel `ETHZ_Database_Access_tool.xlsm` с макросами. При разборе выяснилось (D-027):

- все листы самой книги пусты, это только интерфейс;
- код макросов (извлечён утилитой olevba) показывает форму фильтров, затем открывает соседний файл `Support_file.xlsx`, копирует подходящие строки из листа с данными и строит статистику;
- логика выборки воспроизведена на Python (`scripts/load_ethz.py`), без Excel и макросов. Заодно забираются поля, которые использует инструмент: тип исходного события, система, компонент, цепочка отказов.

Для подбора аналогов оставлены 783 события на энергоблоках с реакторами PWR, как в NPPAD (D-020). Из них у 352 размечен тип исходного события: Transient — 124, LOOP (потеря внешнего питания) — 108, потеря основной питательной воды — 33, разрыв трубки ПГ — 29, малая течь — 23 и т. д.

6.5. Variable_Power: аварии при изменении мощности

В том же репозитории NPPAD есть 12 прогонов, где мощность меняется между 70 и 100 %: четыре нормальных изменения мощности и восемь аварий на их фоне. Они были только в формате .mdb; их прочитали на чистом Python (пакет access-parser) и сконвертировали в CSV (D-029).

В обучение эти прогоны не входят. Это внешний стресс-тест: обучающие данные NPPAD все начинаются со 100 % мощности, и интересно, как система поведёт себя в непривычном режиме. Правильное поведение — не путать плановое изменение мощности с аварией и не ставить уверенных диагнозов там, где режим незнаком.

7. Этап A: модель данных

Цель этапа — подготовить данные и «общий язык» для всех будущих модулей. Подробно — `docs/stage-A-data-model.md`.

7.1. Реестр каналов

Все 96 каналов описаны в файле конфигурации `config/channels.toml`: имя, описание, единица измерения, подсистема станции. Подсистемы: первый контур (19 каналов), второй контур (15), радиационный контроль (13), активная зона (10), топливо (7), защитная оболочка (7), системы аварийного охлаждения (6), тяжёлая авария (6), компенсатор давления (5).

Каналы, которые «подсказывают ответ» (D-007). Восемь каналов описывают саму неисправность: расход течи, энергию течи, течь трубок парогенератора и т. п. На реальной станции таких «приборов» нет, а для системы это фактически готовый ответ. Они помечены `label_proxy` и запрещены и для генератора, и для верификатора. Иначе проверка стала бы нечестной: верификатор «узнавал» бы аварию по прибору «расход течи».

После исключения этих 8 каналов и 7 неизменных остаётся **82 наблюдаемых канала**: с ними работает верификатор.

7.2. Шесть разрешённых каналов генератора

По требованию 4 генератор видит только ограниченный набор каналов. Выбраны шесть: по одному на каждую **критическую функцию безопасности** PWR. Это шесть вещей, которые оператор контролирует в первую очередь (D-006):

Функция безопасности	Канал	Что измеряет
подкритичность (реактор под контролем)	PWNT	мощность по нейтронному потоку
охлаждение активной зоны	TAVG	средняя температура теплоносителя

Функция безопасности	Канал	Что измеряет
теплоотвод	PSGA	давление в парогенераторе A
целостность первого контура	P	давление в первом контуре
защитная оболочка	PRB	давление в здании реактора
запас теплоносителя	LVPZ	уровень в компенсаторе давления

Набор не подбирался «под точность» и намеренно неполон. Например, разрывы трубок в парогенераторах A и B по этим шести каналам почти не отличить. В этом смысл архитектуры: гипотеза строится по части данных («с первого взгляда»), а проверяется по всем. Верификатор видит, в каком парогенераторе растёт радиация, и разводит гипотезы.

7.3. Как готовятся данные

Одна команда `uv run python scripts/load_nppad.py` (около 15 секунд):

1) читает каждый прогон и расставляет столбцы по реестру;

2) проверяет, что шаг времени везде ровно 10 секунд;

3) даёт каждому прогону **непрозрачный идентификатор** — хэш вместо пути к файлу (D-012). Путь `LOCA/12.csv` содержит ответ, и если бы он попал в идентификатор, модель могла бы «подсмотреть»;

4) записывает метки (сценарий, тяжесть) в **отдельный файл**, к которому имеет доступ только оценщик;

5) делит прогоны на обучающие, валидационные и тестовые: 60/20/20, поровну по каждому сценарию, со стабильным «зерном» случайности 42, чтобы деление всегда было одинаковым (D-011). Делятся именно прогоны, а не отдельные моменты времени: иначе соседние, почти одинаковые моменты попали бы и в обучение, и в проверку;

6) **нормализует** каналы — приводит к общей шкале (сколько стандартных отклонений от среднего). Среднее и разброс считаются только по

обучающим прогонам: так данные проверки ни в каком виде не влияют на настройку;

7) сохраняет данные и **манифест** — файл с версией источника, хэшем конфигурации и составом выборок. По нему можно доказать, на каких именно данных получен результат.

Три выборки — главный принцип честной оценки:

- **обучающая (train)** — на ней система учится;
- **валидационная (val)** — на ней подбираются настройки и пороги;
- **тестовая (test)** — откладывается до самого конца, на ней получаются итоговые числа.

Для 11 классов получилось 667 обучающих, 222 валидационных и 222 тестовых прогона.

7.4. Окна наблюдений

Система смотрит на станцию через «окно» — отрезок в 12 точек (2 минуты), который сдвигается каждую минуту (D-008). Окно содержит значения всех каналов. Всего около 80 тысяч окон.

7.5. Контракты данных

Контракт — строгое описание того, какие данные передаются между модулями и какие правила для них всегда выполняются. Например: уверенность гипотезы всегда от 0 до 1, гипотезы упорядочены по уверенности, а их сумма не больше 1. Если правило нарушено, программа сразу сообщает об ошибке, а не передаёт мусор дальше.

Главные контракты диагностики:

- **ObservationWindow** — окно по всем каналам;
- **RestrictedWindow** — окно только по шести разрешённым каналам, без метки, времени и идентификатора прогона; создать его может только барьер;

– **ScenarioHypothesis** и **HypothesisSet** — гипотеза и набор из нескольких гипотез (защита от опасностей H3 и H4);
– **ChannelEvidence** — свидетельство «за» или «против» со ссылкой вида `<окно>#<канал>`;
– **EventAnalogue** — похожее историческое событие;
– **HypothesisVerdict** — итог проверки гипотезы;
– **DiagnosticReport** (появился на этапе C) — полный итог диагностики окна.

Для проверки ответа взяты типы из проекта 7-го семестра без изменения смысла: Claim, EvidenceRef, Rule, RulePack, PolicyDecision, VerificationBundle. К видам свидетельств добавлены «телеметрия» и «историческое событие».

7.6. Семантическая модель (граф знаний)

Требование 3 — единая модель диагностической информации. Она сделана как граф: узлы (канал, окно, сценарий, гипотеза, свидетельство, историческое событие, документ, фрагмент, утверждение, правило, решение) и связи между ними («окно наблюдает канал», «свидетельство поддерживает гипотезу», «решение применяет пакет правил»). Связь можно добавить только между допустимыми типами узлов. Попытка связать, например, правило с окном вызывает ошибку. У каждого узла записано происхождение: откуда он взялся, какой версии, когда.

По графу можно пройти путь «данные → гипотеза → проверка → ответ → решение». Отдельную графовую базу данных не использовали: узлов тысячи, а проверяемая схема важнее масштабирования (D-009).

7.7. Информационный барьер

Требование 5: генератор не должен видеть скрытых данных и правильного ответа. Барьер устроен в три слоя:

1) **Тип данных.** Генератор физически может принять только `RestrictedWindow`, где нет полей метки, времени и прогона.

2) **Проекция.** Барьер оставляет только шесть каналов из конфигурации. Каналы-подсказки запрещены уже при чтении конфигурации.

3) **Идентификаторы и метки.** Идентификаторы — хэши без имени сценария, метки лежат в отдельном файле. Специальный тест проверяет, что код генератора даже не импортирует модуль меток.

Каждый слой проверяется тестами, в том числе на реальных данных.

7.8. Итог этапа A

– NPPAD загружается одной командой — да.
– Тест подтверждает, что генератор не видит скрытых каналов и меток — да.
– Тесты базового ассистента проходят — да: 30 из 30 тестов, включая старые.

8. Этап B: методы

На этапе B построены три модуля: верификатор ответа, генератор гипотез и верификатор гипотез с подбором аналогов. Подробно — `docs/stage-B-methods.md`. Числа этого раздела получены на валидационной выборке для 12 классов (до аудита данных); итоговые числа — в разделе 11.

8.1. Верификатор ответа (B3): правила вместо «второго мнения» модели

Это реализация проекта 7-го семестра, ответ на главное замечание. Модуль `atomic/verification/` стоит между агентами и пользователем.

Как устроено. Каждый агент теперь кладёт в свой ответ не только текст, но и список утверждений со ссылками:

– поисковый агент: «ответ основан на фрагментах `mem:1`, `mem:3`»;
– агент выполнения кода: «результат запуска `exec:…`»; для этого исполнитель кода стал записывать каждый запуск в журнал под своим номером;
– агент-аналитик: «рассуждение модели», без внешних источников.

Затем верификатор:

1) назначает каждому источнику **доверие** по его виду. Доверие задаёт политика, а не агент, чтобы агент не мог «завысить» свою уверенность (D-015): подтверждение человека — 1,0, нормативный документ — 0,95, телеметрия — 0,9, результат инструмента — 0,8, фрагмент базы знаний — 0,7, историческое событие — 0,6, текст модели — 0,2;

2) считает силу каждого утверждения как **минимум** доверия по его источникам, а уверенность ответа — как минимум по утверждениям. Ответ не может быть надёжнее своего самого слабого основания;

3) проверяет **правила**. В первой версии пакета (`aes-core-1.0.0`) их три:

- у каждого утверждения есть ссылка (иначе блокировка);

- нет утверждений, опирающихся только на текст модели (иначе пометка);
- каждая ссылка указывает на существующий артефакт (иначе блокировка);

4) принимает **решение**: нарушено правило уровня «ошибка» — BLOCK (вместо ответа безопасная заглушка); нарушено «предупреждение» или уверенность ниже 0,5 — REQUIRE_HUMAN (ответ с пометкой «требуется оценка оператора» и причинами); иначе ALLOW.

Всё вместе (утверждения, результаты правил, решение, версия пакета правил) складывается в **VerificationBundle** и пишется в трассу Langfuse: потом можно восстановить, почему ответ был выпущен.

Как не сломать старый ассистент (D-014). Если сразу включить блокировку в обычном чате, ответы аналитика начнут приходить с пометками, и поведение 6-го семестра изменится, а требование 1 это запрещает. Поэтому введены режимы:

– `off` — проверки нет;
– `shadow` (по умолчанию для чата) — решение вычисляется и записывается в трассу, но текст не меняется;
– `enforce` — решения применяются: заглушка, пометка; в память диалога идёт только выпущенный текст.

Документ «Upgrade» прямо рекомендует вводить новые защитные слои сначала в теневом режиме. Диагностический ответ работает только в `enforce`.

Проверка на живых моделях. «Выполни код print(1+1)» → ALLOW (уверенность 0,8, ссылка на запуск). «Найди информацию об atomic» → ALLOW (0,7, ссылка на фрагмент). «Проанализируй архитектуру AI-агентов» → REQUIRE_HUMAN (0,2: только мнение модели). Так и задумано: рассуждение модели без источников оператор должен оценить сам.

Попутно выяснилось, что модульные тесты шли 22 секунды вместо 0,7: каждый тест пытался отправить трассы в незапущенный Langfuse и ждал таймаута. Теперь тесты трассы не отправляют (D-018).

8.2. Генератор гипотез (B1)

Задача. По окну шести разрешённых каналов выдать несколько гипотез вида «вероятна ситуация X, уверенность Y».

Как. Для каждого из шести каналов считаются семь простых характеристик окна: значение в начале и в конце, среднее, разброс, минимум, максимум и наклон тренда. Всего 42 числа. По ним классификатор (программа, которая по примерам учится относить данные к классам) выдаёт вероятность каждого сценария. Сравнивали два классификатора из библиотеки scikit-learn: логистическую регрессию (простая линейная модель) и градиентный бустинг (ансамбль решающих деревьев). Языковую модель здесь сознательно не использовали: она недетерминирована, плохо калибруется и противоречит идее «модель только помогает с текстом» (D-019).

Калибровка. Сырой бустинг был сверхуверенным: говорил «90 %» там, где прав реже. Применили температурное масштабирование: все вероятности «смягчаются» одним параметром T, подобранным на валидационной выборке. Получилось T = 1,67, и ошибка калибровки (ECE) уменьшилась почти вдвое: с 0,055 до 0,029.

Результат на валидации. Бустинг: правильный сценарий на первом месте в 78,6 % окон, среди трёх гипотез — в 99,8 %. Логистическая регрессия заметно хуже (65 % и 98 %).

Генератор ошибается предсказуемо, в парах, которые по шести каналам почти неразличимы: течь в горячей и холодной нитке (LOCA ↔ LOCAC), разрывы трубок ПГ A и B, извлечение и введение стержней, SLBOC и MD. Правильный ответ почти всегда есть среди трёх гипотез, поэтому главная задача верификатора — выбрать среди них, а не искать ответ заново.

8.3. Верификатор гипотез (B2): независимая проверка по всем каналам

Чем он отличается от генератора. Генератор видит 6 каналов и учится на примерах. Верификатор видит 82 канала и не обучает никаких весов: он сравнивает данные с «сигнатурами» сценариев. Это два независимых взгляда.

Сигнатуры. Для каждого сценария и каждого из 82 каналов по обучающим окнам посчитано, в каких пределах обычно лежат уровень и тренд канала: типичный диапазон (от 5 до 95 процентиля) и допустимый (от 1 до 99 процентиля).

Проверка гипотезы. Чтобы проверить гипотезу «это LOCA»:

1) берутся конкуренты — две другие гипотезы того же набора (контрастная проверка, D-023);

2) выбираются каналы, которые лучше всего отличают LOCA именно от этих конкурентов;

3) по каждому выбранному каналу: значение в типичном диапазоне LOCA — свидетельство «за», за пределами допустимого — «против», между ними — слабое «против»;

4) считается доля подтверждений; при доле выше порога гипотеза **подтверждена**, при доле ниже нижнего порога **отклонена**, иначе **не определено**.

Каждое свидетельство — конкретная проверяемая фраза со ссылкой, например: «Rad Monitor RB air (CPM): уровень −0,37σ — вне допустимого диапазона для LOCAC [+0,18; +5,52]».

Первая версия была слишком мягкой. При исходных настройках верификатор подтверждал 30 % неверных гипотез, то есть был почти бесполезен как защита. Перебрали 24 сочетания настроек на валидации с явным критерием безопасности: максимум подтверждённых верных гипотез при условии, что неверных подтверждено не больше 10 % (D-022). Ложное подтверждение для АЭС опаснее, чем «не определено»: неопределённость оставляет решение оператору, а ложное подтверждение усиливает чрезмерное

доверие автоматике. После подбора неверных подтверждалось 4 %, верных — 73 %.

Что показала промежуточная оценка:

- неверная первая гипотеза подтверждалась лишь в 1 % окон, а без проверки оператор видел бы неверную первую гипотезу в 21 % окон;
- пары, которые генератор путает, верификатор во многом разводит: SLBOC с 27 % до 97 % правильных;
- **течь в горячей и холодной нитке не различалась вообще**: обе гипотезы получали «не определено». Это честный отказ вместо ложной уверенности, но проблема осталась;
- **ситуации вне обучения подтверждались слишком часто**: в 69 % окон хотя бы одна гипотеза «подтверждалась». Верификатор умеет проверить согласие с гипотезой, но не умеет сказать «это вообще ни на что не похоже».

Две последние проблемы стали главными задачами этапа C.

8.4. Аналоги из базы ETHZ

Для каждого сценария NPPAD в конфигурации задан английский поисковый запрос и список систем станции. Поиск — алгоритм **BM25** (классический поиск по словам с учётом их редкости), реализованный в проекте без внешних библиотек (`atomic/memory/bm25.py`). Он же заменил поиск по подстроке в упрощённой памяти ассистента, что закрыло расхождение с отчётом 6-го семестра (D-026).

Аналог подаётся как справка: «В аналогичных случаях наблюдалось: …» — без указаний, что делать. Примеры: для разрыва трубки ПГ — Indian Point 1988, McGuire 1989; для разрыва питательного трубопровода — Loviisa 1993, Mihama 2004.

8.5. Итог этапа B

60 тестов без языковой модели проходят, регрессия старого ассистента тоже. Все критерии готовности выполнены: у каждого модуля есть тесты, барьер проверен, верификатор ответа блокирует ответ без ссылок и пропускает ответ с разрешимыми ссылками. Тег `v0.3-stage-b`.

9. Этап C: интеграция, эксперименты и исправление слабых мест

Этап C по плану — соединить диагностику с ассистентом и провести эксперименты. Перед этим нужно было ответить на пять вопросов, поставленных по итогам этапа B: про файл ETHZ, про вопросы оппонентов на GitHub, про различение течей, про слишком частое подтверждение и про однопрогонные классы. Ответы на три из них уже даны в разделах 5.3, 6.3 и 6.4. Ниже — остальное. Подробно — `docs/stage-C-integration.md`.

9.1. Как научились различать течь в горячей и холодной нитке

Проблема. В одном двухминутном окне эти две течи выглядят почти одинаково. Отдельный эксперимент: классификатор по всем 574 характеристикам окна различал их с точностью 54 % — почти как подбрасывание монеты.

Наблюдение. Различие есть, но оно в **начале** события: в первые 30–120 секунд процессы идут по-разному. Классификатор по окнам первых 300 секунд различал течи уже в 88,5 % случаев. А по изменениям каналов через 30, 60 и 120 секунд от начала события 40 валидационных прогонов разделились **без единой ошибки**. Проверили, что это не случайность: результат сохраняется при добавлении шума 5 % и даже если убрать восемь самых информативных каналов. Значит, различие физическое, а не артефакт одного прибора.

Решение (D-030). Верификатор получил **память о событии**: исходное состояние до события и показания первых 120 секунд после начала. Память строго «причинная»: в неё попадает только то, что уже произошло к текущему моменту. Будущее система не видит. По этой памяти строятся сигнатуры ранней фазы: насколько изменился каждый из 82 каналов через 30, 60 и 120 секунд.

Генератор при этом не изменился: он по-прежнему видит только текущее окно шести каналов, и барьер не нарушен.

Первый итоговый прогон и доработка (D-038). На тестовой выборке LOCA всё ещё в половине окон получала исход «неоднозначно». Разбор на валидации показал, что это малые течи: признаки, выбранные «в среднем против всех конкурентов», отличали LOCA от разрыва трубки ПГ, но не от LOCAC. Исправление: для каждого конкурента выбирать свои признаки, которые лучше всего отличают гипотезу именно от него (попарный отбор). После этого LOCA определяется в 96 % окон, ошибочных определений нет.

Здесь же честная оговорка. Проблему заметили на тестовой выборке, а исправление разрабатывали и настраивали только на валидационной. Поэтому итоговые числа на тесте не полностью «нетронутые». Оба прогона (до и после) сохранены, а это отмечено в документах.

9.2. Как перестали подтверждать незнакомые ситуации

Добавлены две проверки, которых верификатору не хватало (D-032).

Есть ли вообще отклонение. Если ни один наблюдаемый канал не отличается от исходного состояния больше чем на 1 %, ответ — «отклонений не обнаружено». Четыре канала дрейфуют даже в спокойной станции; это установили по прогонам без события, и их не учитывают. Сначала порог был «больше четырёх отклонившихся каналов», и он пропускал MD и SLBOC. При разборе этой проблемы и был обнаружен «пустой» класс MD (раздел 6.2).

Похожа ли ситуация на известные. Система ищет самое похожее событие из обучающих (по ранней фазе) и самое похожее окно (по текущему состоянию). Если даже самое похожее слишком далеко, ответ — «ситуация не похожа ни на один известный сценарий». Порог подобран на валидации так, чтобы на известных сценариях ложная тревога возникала примерно в 2 % окон.

Итоговое правило выбора (D-033). Теперь каждое окно получает один из пяти исходов:

1) нет отклонения — «отклонений не обнаружено»;

2) событие новое — «незнакомая ситуация»;

3) ни одна гипотеза не подтверждена — «не подтверждено»;

4) подтверждена одна гипотеза, или лидер заметно (на 0,1 по доле подтверждений) опережает вторую — «вероятна ситуация X»;

5) иначе — «неоднозначно».

Результат. Все семь проб вне обучения получают ожидаемый исход в 100 % окон, и ни одного ложного диагноза. На этапе B подтверждения были в 69 % окон.

Предел метода. Проверили, что будет, если сценарий действительно новый, но физически похож на известный. Каждый класс по очереди убирали из эталона и смотрели, распознается ли он как новый. В среднем — только в 43 % окон. Хорошо отделяются LLB, LR, RI, SLBIC (89–100 %), почти не отделяются LOCA и LOCAC (каждая похожа на другую), SGATR, RW, SLBOC (0–12 %). Это свойство метода, а не настройки: если новое похоже на старое, его трудно отличить по данным. Это записано в ограничениях.

9.3. Соединение с ассистентом

Диагностический ответ (D-036). `AnalystAgent` получил метод `explain_diagnosis`. Он строит ответ оператору из итогов диагностики по строгому шаблону:

- оценка ситуации, например «Вероятна ситуация LOCA: подтверждена независимой проверкой, 100 % взвешенных свидетельств „за“»;
- все гипотезы со статусами проверки;
- свидетельства [E1], [E2]… со ссылками на окно и канал;
- аналоги из ETHZ [A1]… со ссылками на событие;
- справка из базы знаний [D1] (добавлена на этапе E);
- обязательная фраза: «Система только интерпретирует данные; решение о действиях принимает оператор».

Каждая фраза ответа — утверждение со ссылками, которое проверяет верификатор ответа. Ссылки на окна проверяет источник телеметрии, на события — база ETHZ.

Пакет правил aes-core-1.1.0 (D-035). К трём правилам добавлены два:

– диагноз выпускается без оговорки только при исходах «вероятна ситуация X» и «отклонений нет»; «неоднозначно», «не подтверждено» и «незнакомая ситуация» идут с пометкой для оператора;
– в ответе нет указаний к действию: «откройте», «необходимо снизить», «рекомендуется» и т. п. приводят к блокировке. Это прямое воплощение инварианта «Upgrade».

Необязательное пояснение языковой модели. По запросу языковая модель может добавить короткое пояснение человеческим языком. Оно разбирается по предложениям: предложение со ссылками [E…] опирается на них, предложение без ссылок — только на модель (это даёт пометку), ссылка на несуществующее свидетельство блокирует.

Точка входа. `AtomicWorkflow.diagnose(source, t)` берёт окно телеметрии на момент t, прогоняет всю диагностику, строит ответ и проверяет его всегда в режиме `enforce`. Источник телеметрии (`TelemetryReplay`) только читает данные, и управлять через него ничем нельзя. Через API — `POST /diagnose {run_id, t}`.

Трассировка (требование 10). В Langfuse пишется трасса `atomic.diagnose` с отдельными шагами: гипотезы генератора, проверка, новизна, справка, итоговое решение и полный VerificationBundle. По трассе восстанавливается путь «данные → гипотеза → проверка → ответ → решение».

9.4. Оценка качества (benchmark)

Отдельный скрипт `scripts/benchmark.py` — единственная часть системы, которая видит правильные ответы. Он прогоняет все тестовые окна и считает метрики: точность генератора и калибровку, качество проверки, распределение исходов, поведение на пробах и на стресс-тесте, работу верификатора ответа на нормальных и специально испорченных ответах,

качество поиска аналогов, скорость. К каждому результату прикладывается **паспорт версий**: версии моделей, хэши конфигурации и списка зависимостей, версия Python и библиотек, коммит данных. С флагом `--repeat` оценка выполняется дважды, и проверяется, что числа совпали.

9.5. Найденная регрессия на стресс-тесте и её исправление (D-039)

После попарного отбора признаков на стресс-тесте Variable_Power появилась ошибка: нормальное снижение мощности со 100 до 80 % в 18 % окон «распознавалось» как сброс нагрузки (LR). Физически это объяснимо (плавное снижение нагрузки похоже на её сброс), но для системы это ложный диагноз.

Причина: в стресс-тесте начало события отсчитывалось от начала записи, а снижение мощности начинается примерно через 420 секунд. Память о событии описывала спокойный стационар, и проверка новизны ничего не замечала. Для аварийных прогонов начало бралось из описания («неисправность введена в 915 с»). Это знание, которого в реальной эксплуатации нет.

Исправление: начало события определяется по самим данным. Точка «стационарна», если за последние 60 секунд ни один канал не изменился больше чем на 1 %, а начало события — последняя стационарная точка перед отклонением. После этого ложных диагнозов на стресс-тесте не осталось ни до, ни после ввода неисправности. Нормальные изменения мощности и аварии на их фоне получают исход «незнакомая ситуация»: такого режима в обучающих данных нет, и система честно об этом говорит. На всех 222 тестовых прогонах NPPAD детектор находит начало там же, где оно задано в данных, так что обучение не меняется.

9.6. Оценка пояснений языковой модели и откат (D-041)

Пояснения проверили на реальной модели qwen2.5-7B: по одному окну каждого из 11 классов и двух проб. Результат первого прогона: 79 %

предложений со ссылками, указаний к действию нет, решения — 4 ALLOW, 7 REQUIRE_HUMAN, 2 BLOCK. Блокировки были правильными: в пробах модель сослалась на несуществующие свидетельства.

Но из-за плохого пояснения блокировался **весь** ответ, включая корректную детерминированную часть. Оператор терял проверенную диагностику из-за ошибки необязательного компонента. Поэтому добавлен откат: если ответ с пояснением получает BLOCK, пояснение отбрасывается, детерминированная часть проверяется заново и выпускается с пометкой «Пояснение языковой модели не выпущено…». Отклонённое пояснение сохраняется в трассе для аудита. Во втором прогоне: 11 ALLOW, 2 REQUIRE_HUMAN, 0 BLOCK, одно пояснение отброшено.

Заметили и другое: ответы модели сильно меняются от запуска к запуску даже при одинаковом входе. Воспроизводима только детерминированная часть ответа, а пояснение — нет.

9.7. Итог этапа C

Тег `v0.4-stage-c`. Сквозной сценарий «окно NPPAD → гипотезы → проверка → ответ → решение» выполняется автоматически, повторный прогон даёт те же числа — оба критерия готовности выполнены.

10. Этап E: доводка, проверка воспроизводимости и заморозка

По дорожной карте этап E (январь–февраль 2027) — исправить дефекты, найденные в экспериментах, дописать тесты, провести финальный прогон, проверить воспроизводимость «с нуля» и заморозить версию. Его выполнили сразу после этапа C, чтобы писать отчёты на окончательных числах. Подробно — `docs/stage-E-freeze.md`.

10.1. Ревизия кода и найденные дефекты (D-044)

Код просмотрели целиком и прогнали через линтер (программу, которая ищет подозрительные места). Найдено и исправлено:

1) **Данные из будущего.** Если запросить диагноз в момент t раньше конца первого окна (например, на 50-й секунде, когда первое двухминутное окно ещё не закончилось), источник телеметрии возвращал это первое окно. То есть показывал данные позже момента t. Для системы, которая должна работать «в реальном времени», это недопустимо. Теперь возвращается понятная ошибка «первое окно завершается в 110 с», а API отвечает кодом 422.

2) **Ошибки веб-интерфейса.** Запрос с несуществующим прогоном приводил к внутренней ошибке сервера (500), теперь отвечает «не найдено» (404). Исход диагноза брался из общей переменной, которую могут перезаписать параллельные запросы, то есть ответ одному пользователю мог получить чужой исход. Теперь исход берётся из самого ответа. Набор данных раньше загружался заново на каждый запрос, теперь один раз.

3) **Молчаливое усечение.** В нескольких местах две последовательности, которые обязаны быть одной длины, объединялись без проверки. Если бы длины разошлись, лишнее молча отбросилось бы. Теперь это ошибка.

4) **Замкнутая оценка аналогов.** Совмещённый режим поиска аналогов использует то же поле «тип исходного события», по которому размечена релевантность. Поэтому его высокая точность частично получается «по

построению». Честной мерой качества поиска считается текстовый режим; это записано в документах.

5) **Неточность в документе этапа C.** Там было написано «точность 1,0 у всех классов». На самом деле у RW она 0,974, у SLBIC 0,999. Исправлено.

10.2. Защёлка события (D-042)

Детектор начала события из этапа C (раздел 9.5) решал проблему стресс-теста, но при проверке на обычных данных обнаружился его недостаток. После аварии станция часто выходит на новое стационарное состояние: давления и температуры перестают меняться, но остаются ненормальными. Детектор считал такую точку «стационарной», начинал отсчёт заново и сравнивал дальнейшее уже с аварийным состоянием. В итоге выходило «отклонений нет» при развившейся аварии. На валидации так происходило в **25,6 %** окон (у разрыва линии продувки — в 77 %).

Исправление — **защёлка**. Событие считается законченным, только когда станция стоит в стационаре 120 секунд подряд **и** вернулась к состоянию до события. Новое ненормальное равновесие событие не закрывает. Доля ошибочно закрытых событий упала до **1,1 %** (остался только SLBOC, у которого теплогидравлика и правда возвращается к норме).

История порога. Сначала был выбран порог 300 секунд стационара. Финальный прогон показал 12,8 % неверных диагнозов на стресс-тесте. В файлах $100 \to 80$ % в начале записи есть короткий пусковой процесс симулятора, станция успокаивается, но до начала снижения мощности стоит в стационаре только около 250 секунд, а не 300. Событие не закрывалось, и память о начале описывала пусковой процесс. Порог заменили на 120 секунд: это ровно горизонт, по которому оценивается начало события, то есть естественная величина. На валидации оба порога дают одинаковый результат (1,13 % и 1,09 %), а порог 60 секунд уже ломает другой класс (FLB). Честная оговорка: проблему увидели на стресс-тесте, а значение проверили на

валидации. Поэтому результаты стресс-теста после исправления — не полностью независимая оценка.

Теперь причинный детектор используется везде: в реальном пути диагностики (`TelemetryReplay → diagnose`), на стресс-тесте и в отдельном блоке оценки. Знание «когда введена неисправность» нигде не используется. На тестовой выборке результаты с детектором и без него совпали полностью.

10.3. Справка по сценариям в базе знаний (D-043)

Дорожная карта требует, чтобы объяснение ссылалось на документацию. Механизм ссылок [D…] на фрагменты базы знаний был сделан ещё на этапе C, но база знаний ассистента содержала только инструкции к ИТ-системе.

Решение без выдумывания: у каждого прогона NPPAD есть журнал событий симулятора («Transient Report»). Там записано, в какой момент сработала аварийная защита, когда включились насосы аварийного охлаждения, когда закрылись клапаны. Скрипт `scripts/build_scenario_kb.py` собирает по журналам **обучающих** прогонов справку для каждого сценария: какие действия систем станции происходят, в какой доле прогонов и когда (медиана и разброс). Пример для течи в горячей нитке (60 прогонов):

- сигнал аварийной защиты по низкому давлению первого контура — 95 % прогонов, медиана 125 с;
- аварийная остановка реактора — 100 %, 126 с;
- включение насосов высоконапорного впрыска — 100 %, 133 с;
- запуск спринклерной системы защитной оболочки — 100 %, 159 с;
- оголение активной зоны — 47 %, 16 мин.

Справка описательная, без указаний к действию. В ответ она попадает только для установленного сценария, со ссылкой [D1], которую проверяет верификатор. В финальной оценке справка приложена к 533 из 560 ответов, то

есть ко всем с исходом «вероятна ситуация X». Это описание поведения модели станции, а не эксплуатационная документация реальной АЭС.

10.4. Оценка пояснений «моделью-судьёй» (LLM-as-a-Judge)

Как и в 6-м семестре, качество текста оценивает другая языковая модель через библиотеку deepeval. Судья — модель другого семейства (vikhr-llama3.1), чем автор пояснений (qwen2.5), чтобы модель не оценивала саму себя. Метрики: **верность** (не противоречит ли пояснение отчёту) и **релевантность** (отвечает ли на вопрос). Чтобы понять, можно ли доверять самому судье, добавлены две контрольные пары: заведомо верное пояснение и заведомо искажённое («давление упало до нуля» при росте давления).

Результаты: средняя верность 0,71, релевантность 0,76. Контроль: по верности судья отличает верное пояснение от искажённого (0,67 против 0,33), но шкала у него грубая, и даже верное он оценил всего в 0,67. Релевантность на правдивость не реагирует: искажённое пояснение получило 1,0. Вывод: судья — грубый вспомогательный инструмент, безопасность держится на детерминированных правилах. Так и было задумано в дорожной карте.

10.5. Прочее

- **Поиск аналогов.** Добавлена метрика recall@k (какая доля всех подходящих событий найдена в первых k): текстовый поиск находит 35 % подходящих событий в первых 20 при точности 0,68 в первых трёх.
- **Версии моделей.** В паспорт результатов теперь пишется не только имя модели Ollama, но и её цифровой отпечаток (digest): одно имя `model:latest` может со временем указывать на разные веса.
- **Живая трасса Langfuse.** Подняли локальный Langfuse, выполнили диагностику и прочитали трассу обратно через API. По ней восстанавливается весь путь от окна телеметрии до решения ALLOW. Выгрузка сохранена как свидетельство.

10.6. Воспроизводимость «с нуля»

Самая строгая проверка: свежая копия репозитория в отдельной папке, установка зависимостей строго по зафиксированному списку (`uv sync --frozen`), подготовка данных и **обучение всех моделей заново**, затем оценка. Результат сравнили с основным прогоном специальным скриптом: метрики и паспорт версий совпали полностью. Справка по сценариям собралась байт-в-байт как в git.

По пути нашёлся ещё один пробел: версия Python не была закреплена, и чистая копия собралась на Python 3.14 вместо 3.12. Числа совпали и так, это хороший знак устойчивости, но версию закрепили (`.python-version`). Повторная сборка на 3.12 совпала полностью, включая паспорт.

10.7. Заморозка

Тег `v1.0-freeze`. По правилу дорожной карты после него код меняется только для исправления ошибок. Все итоговые числа для ВКР — в `docs/test-results/2026-10-25-benchmark-final.json`.

11. Итоговые результаты и что они означают

Все числа — на тестовой выборке (13 984 окна из 222 прогонов, 11 классов), которую система не видела ни при обучении, ни при подборе настроек.

11.1. Генерация гипотез

Показатель	Значение	Что это значит
верный сценарий первым	0,888	по шести «главным» приборам генератор сразу угадывает в 89 % окон
верный сценарий среди трёх гипотез	0,996	правильный ответ почти всегда есть в списке, который получает верификатор
ошибка калибровки до / после	0,048 / 0,028	после калибровки «уверенность 80 %» ближе к реальным 80 % правоты

11.2. Проверка гипотез

Показатель	Значение	Что это значит
верных гипотез подтверждено	0,969	верную гипотезу верификатор почти всегда подтверждает
неверных гипотез подтверждено	0,015	неверную подтверждает редко, в 1,5 % случаев
верный сценарий на первом месте после проверки	0,994	проверка поднимает верную гипотезу наверх
то же без памяти о событии	0,847	память о начале события даёт +15 процентных пунктов

11.3. Итоговые исходы

Показатель	Значение
доля окон с диагнозом «вероятна ситуация X»	0,952
точность этого диагноза	0,999
неверный диагноз, от всех окон	0,001 (примерно 1 окно из 1000)
«незнакомая ситуация» на известных сценариях (ложная тревога)	0,014
«не подтверждено»	0,031
«неоднозначно»	0,003
без памяти о событии: доля диагнозов / точность	0,623 / 0,985

По классам доля диагнозов от 0,88 (RW) до 0,99 (SGBTR). Неверные диагнозы есть только у RW (точность 0,974) и SLBIC (0,999), у остальных девяти классов точность 1,0. Течь в горячей нитке (LOCA) диагностируется в 96 % окон, в холодной (LOCAC) — тоже в 96 %, путаницы между ними нет.

11.4. Ситуации вне обучения

Проба	Что это	Ожидается	Получено
ATWS, MD, SP	события нет	«отклонений нет»	100 % окон
LACP, LOF, Normal, TT	сценарии, которых не было в обучении	«незнакомая ситуация»	100 % окон

Ни одного ложного диагноза. На этапе B подтверждения на таких пробах были в 69 % окон.

11.5. Стресс-тест: изменение мощности

Ни одного ложного диагноза до ввода неисправности и ни одного неверного после. Правильных диагнозов на стресс-тесте тоже нет: система честно отвечает «незнакомая ситуация», потому что в обучающих данных не было режимов с другой мощностью. Для системы поддержки оператора это правильное поведение: лучше сказать «не знаю», чем уверенно ошибиться.

11.6. Проверка ответов

Показатель	Значение
проверено ответов	560
ALLOW / REQUIRE_HUMAN / BLOCK	95,2 % / 4,8 % / 0 %
испорченных ответов (без ссылок, ссылка «в никуда», указание к действию)	1680
из них выпущено как ALLOW	0

Все пометки REQUIRE_HUMAN приходятся именно на неуверенные исходы: «не подтверждено», «незнакомая ситуация», «неоднозначно».

11.7. Скорость и воспроизводимость

– Весь диагностический контур без языковой модели — медиана 26 мс на окно (95 % окон быстрее 43 мс). Данные приходят раз в 10 секунд, так что запас огромный. Пояснение языковой модели занимает десятки секунд, поэтому оно необязательное и по умолчанию выключено.
– Повторный прогон — в точности те же числа. Сборка с нуля на чистой копии — те же числа.

12. Ограничения и честные оговорки

Это нужно для главы «Ограничения» отчёта и ВКР.

1) **Данные из симулятора.** Все данные получены на симуляторе PCTRAN для одного типа PWR, а не на реальной станции. Все прогоны начинаются со 100 % мощности. При другой мощности система отвечает «незнакомая ситуация», а не диагноз.

2) **Похожий новый сценарий не отличить от известного.** Если появится новая авария, физически похожая на известную, проверка новизны поймает её примерно в 43 % окон.

3) **SLBOC различим только по радиации.** Если мониторы радиации откажут, этот класс станет неотличим от нормальной работы.

4) **Плановое изменение мощности** остаётся «событием», пока мощность не вернётся к исходной. Система отвечает «незнакомая ситуация» и просит оценки человека. Это консервативно; кнопки «оператор подтвердил новое исходное состояние» пока нет.

5) **Справка по сценариям** описывает поведение модели станции в симуляторе, а не эксплуатационную документацию.

6) **Тестовая выборка использовалась для поиска двух проблем** на этапе C (неразличимость малых течей, регрессия на стресс-тесте), а стресс-тест — для одной на этапе E. Все исправления проверялись только на валидационной выборке, но итоговые числа на этих наборах — не полностью независимая оценка. Это раскрыто в документах.

7) **Пояснения языковой модели недетерминированы**: один и тот же вход даёт разные тексты. Воспроизводима только детерминированная часть ответа.

8) **Классы с одним прогоном** нельзя использовать для обучения. Улучшить это можно только новыми симуляциями в PCTRAN.

9) **Объём проверки утверждений в обычном чате.** Для поискового агента утверждение одно на весь ответ: проверяется, что найденные

фрагменты существуют, но не то, что модель действительно опиралась на них. Диагностический ответ разбит на утверждения по фразам.

13. Что осталось сделать

Техническая часть закончена и заморожена. Осталась письменная часть:

– **до 31.10.2026** — контрольная точка этапа А, «описание модели данных и методов» (документ подготовлен: `reports/Этап А — описание модели данных и методов.docx`);
– **22–28.12.2026** — отчёт по практике по ГОСТ 7.32 (не меньше 25 страниц, копия задания на второй странице; черновик подготовлен: `reports/Отчёт по производственной практике (ГОСТ 7.32).docx`), отзыв руководителя, презентация;
– **весна 2027** — преддипломная практика, текст ВКР, защита.

Ветка `diploma` и теги пока только на компьютере; на GitHub их можно отправить отдельно.

14. Приложения

Приложение А. Словарь

Термин	Простыми словами
АЭС, PWR	атомная электростанция; PWR — тип реактора, где вода под высоким давлением служит и теплоносителем, и замедлителем (близок к ВВЭР)
первый и второй контур	первый — вода, которая проходит через реактор; второй — вода, которая в парогенераторе превращается в пар для турбины
горячая и холодная нитка	участки первого контура: от реактора к парогенератору (горячая) и обратно (холодная)
критические функции безопасности	шесть вещей, за которыми оператор следит в первую очередь: реактор заглушен, активная зона охлаждается, тепло отводится, первый контур цел, оболочка цела, теплоносителя достаточно
NPPAD	открытый набор данных с показаниями приборов во время симулированных аварий
прогон	одна симуляция одного сценария определённой тяжести
канал	один «прибор» — одна величина во времени (давление, температура и т. п.)
окно	отрезок данных в 2 минуты, на который система смотрит за раз
память о событии	исходное состояние и первые 2 минуты после начала события
обучающая / валидационная / тестовая выборка	данные для обучения, для подбора настроек и для итоговой проверки; тестовую откладывают до конца
нормализация	приведение всех каналов к общей шкале
классификатор	программа, которая по примерам учится относить данные к классам
градиентный бустинг	классификатор из множества простых решающих деревьев, каждое исправляет ошибки предыдущих
калибровка	подстройка, чтобы названная уверенность совпадала с реальной долей правильных ответов
ECE	ошибка калибровки: насколько в среднем названная уверенность отличается от реальной правоты
сигнатура сценария	типичные диапазоны значений каналов для сценария
свидетельство	факт «канал X в окне Y ведёт себя так, как ожидается (или не ожидается) при сценарии Z»
информационный барьер	механизм, не дающий генератору увидеть лишнее

Термин	Простыми словами
label_proxy	канал, который фактически содержит ответ (например, «расход течи»); запрещён
RAG	ответ языковой модели по найденным фрагментам документов
языковая модель (LLM)	нейросеть, которая пишет текст (здесь qwen2.5, deepseek-r1 и др., работают локально через Ollama)
LLM-as-a-Judge	оценка текста другой языковой моделью
BM25	классический алгоритм поиска текста по словам
precision@k, recall@k	точность: какая доля найденного в первых k подходит; полнота: какая доля всего подходящего нашлась в первых k
Claim, EvidenceRef, RulePack, PolicyDecision	утверждение, ссылка на свидетельство, набор правил, решение о выпуске ответа
ALLOW / REQUIRE_HUMAN / BLOCK	выпустить / выпустить с пометкой «требуется оценка оператора» / заблокировать
Langfuse	система записи трасс: что происходило внутри при обработке запроса
benchmark	набор измерений качества системы на отложенных данных
паспорт версий	список версий всего, от чего зависит результат: моделей, данных, конфигурации, библиотек
тег git	постоянная метка версии кода

Приложение Б. Хронология

Дата	Событие	Метка в git
30.03.2026	ассистент 6-го семестра	`v0.1-baseline` (a3f0e16)
24.10.2026	дорожная карта; разбор репозитория; гигиена git	4a3400e
24.10.2026	этап A: данные NPPAD, контракты, граф знаний, барьер	`v0.2-stage-a`
24.10.2026	этап B: верификатор ответа, генератор, верификатор гипотез, аналоги	`v0.3-stage-b`
24.10.2026	удаление вопросов оппонентов из истории	472ba13
24–25.10.2026	этап C: память о событии, аудит данных, новизна, интеграция, benchmark	`v0.4-stage-c`
25.10.2026	очистка GitHub завершена	—
25.10.2026	этап E: дефекты, защёлка, справка, судья, воспроизводимость	`v1.0-freeze`

Приложение В. Где что лежит

Что	Где
код диагностики	`atomic/diagnostics/` (данные, генератор, верификатор, новизна, начало события, конвейер)
контракты данных	`atomic/contracts/`
граф знаний	`atomic/knowledge/`
верификатор ответа	`atomic/verification/`, правила — `config/rulepacks/`
агенты и оркестратор	`atomic/agents/`, `atomic/orchestration/workflow.py`
конфигурация	`config/diagnostics.toml`, `config/channels.toml`, `config/analogues.toml`
справка по сценариям	`dataset/scenarios/`
скрипты	`scripts/` (подготовка данных, обучение, оценка, демонстрация)
тесты	`tests/` (130 быстрых + 2 медленных)
документация	`docs/` (этот отчёт, журнал, решения, документы этапов)
результаты прогонов	`docs/test-results/`
документы для сдачи	`reports/` (.docx) и их исходники `reports/src/`

Приложение Г. Как всё запустить

```
uv sync                                          # зависимости (Python 3.12 закреплён)
git clone --depth 1 https://github.com/thu-inet/NuclearPowerPlantAccidentData.git
NPPAD/raw
uv run python scripts/load_nppad.py              # подготовка NPPAD
uv run python scripts/load_ethz.py               # база событий ETHZ
uv run python scripts/convert_variable_power.py  # стресс-тест изменения мощности
uv run python scripts/train_generator.py         # генератор гипотез
uv run python scripts/train_verifier.py          # сигнатуры верификатора
uv run python scripts/train_novelty.py           # детектор новизны
uv run python scripts/build_scenario_kb.py       # справка по сценариям
uv run python scripts/benchmark.py --repeat      # итоговая оценка
uv run python scripts/demo_diagnosis.py --scenario LOCA --t 300   # пример ответа
оператору
uv run pytest -m "not slow"                      # тесты
```

Если запущен Docker Desktop с контейнером `ollama`, укажите `OLLAMA_HOST=http://127.0.0.1:11434`: иначе запросы уйдут в контейнер, а не в системный Ollama.